```
// g++ -O2 -std=c++17 b_verify_exact.cpp -lgmp -o b_verify_exact
// ./b_verify_exact > b_verification_output.txt
// Exact endpoint certificate for the moment estimates (2.34)-(2.52).
// b_star=13/10000; the preliminary absolute bounds use beta<=1/100.
// All mathematical calculations use GMP exact rational arithmetic.
// Functional integral estimates and the alpha/pi bounds are from the paper.
// Nonnegative polynomial coefficients certify the endpoint reductions.

#include <gmp.h>
#include <chrono>
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

class Q
{
    mpq_t value;

  public:
    Q(long numerator = 0, unsigned long denominator = 1)
    {
        if (denominator == 0)
            throw std::invalid_argument("Zero denominator");
        mpq_init(value);
        mpq_set_si(value, numerator, denominator);
        mpq_canonicalize(value);
    }

    explicit Q(const std::string &text)
    {
        mpq_init(value);
        if (mpq_set_str(value, text.c_str(), 10) != 0)
        {
            mpq_clear(value);
            throw std::invalid_argument("Invalid rational literal");
        }
        mpq_canonicalize(value);
    }

    Q(const Q &other)
    {
        mpq_init(value);
        mpq_set(value, other.value);
    }

    Q(Q &&other) noexcept
    {
        mpq_init(value);
        mpq_swap(value, other.value);
    }

    Q &operator=(Q other) noexcept
    {
        mpq_swap(value, other.value);
        return *this;
    }

    ~Q()
    {
        mpq_clear(value);
    }

    friend Q operator+(const Q &a, const Q &b)
    {
        Q result;
```

```
        mpq_add(result.value, a.value, b.value);
        return result;
    }

    friend Q operator-(const Q &a, const Q &b)
    {
        Q result;
        mpq_sub(result.value, a.value, b.value);
        return result;
    }

    friend Q operator*(const Q &a, const Q &b)
    {
        Q result;
        mpq_mul(result.value, a.value, b.value);
        return result;
    }

    friend Q operator/(const Q &a, const Q &b)
    {
        if (mpq_sgn(b.value) == 0)
            throw std::invalid_argument("Division by zero");
        Q result;
        mpq_div(result.value, a.value, b.value);
        return result;
    }

    friend bool operator<(const Q &a, const Q &b)
    {
        return mpq_cmp(a.value, b.value) < 0;
    }

    friend bool operator==(const Q &a, const Q &b)
    {
        return mpq_cmp(a.value, b.value) == 0;
    }

    friend std::ostream &operator<<(std::ostream &out, const Q &a)
    {
        char *buffer = mpq_get_str(nullptr, 10, a.value);
        std::string text(buffer);
        void (*release)(void *, size_t);
        mp_get_memory_functions(nullptr, nullptr, &release);
        release(buffer, text.size() + 1);
        return out << text;
    }
};

Q power(Q base, unsigned exponent)
{
    Q result(1);
    while (exponent)
    {
        if (exponent & 1U)
            result = result * base;
        base = base * base;
        exponent >>= 1;
    }
    return result;
}

struct Term
{
    Q coefficient;
    unsigned exponent;
};
```

```
int checks = 0, failures = 0, polynomial_checks = 0;

void check(bool condition, const char *equation, const char *statement)
{
    ++checks;
    if (!condition)
        ++failures;
    std::cout << (condition ? "PASS " : "FAIL ") << checks << ' ' << equation
              << ' ' << statement << '\n';
}

// Each nonnegative coefficient times beta^k is nondecreasing for beta>0.
// Therefore the computed endpoint bound holds throughout 0<beta<=endpoint.
void polynomial_bound(const std::vector<Term> &terms, const Q &endpoint,
                      const Q &bound, const char *equation,
                      const char *statement, bool allow_equality = false)
{
    bool monotone = Q(0) < endpoint;
    Q value(0);
    for (const auto &term : terms)
    {
        if (term.coefficient < Q(0))
            monotone = false;
        value = value + term.coefficient * power(endpoint, term.exponent);
    }
    const bool bounded = allow_equality ? !(bound < value) : value < bound;
    check(monotone && bounded, equation, statement);
    ++polynomial_checks;
}

Q factorial(unsigned n)
{
    Q result(1);
    for (unsigned k = 2; k <= n; ++k)
        result = result * Q(k);
    return result;
}

int main()
{
    try
    {
        const auto start = std::chrono::steady_clock::now();
        const Q b(13, 10000), B(1, 100), alpha_low(41, 25), pi_low(157, 50);
        const Q ten21 = power(Q(10), 21);
        std::cout << "ENDPOINTS b_star=" << b << " preliminary_beta_max=" << B
                  << " arithmetic=GMP_exact_rational\n";
        check(Q(0) < b && b < B, "[domain]", "0 < b_star=13/10000 < 1/100");
        check(Q(3, 2) < alpha_low, "[2.1]", "alpha > 41/25 > 3/2");
        check(Q(33, 20) < Q(5, 3), "[2.1]", "alpha < 33/20 < 5/3");
        check(Q(3) < pi_low, "[2.1]", "pi > 157/50 > 3");
        check(Q(3) < power(Q(7, 4), 2), "[2.34]", "sqrt(3) < 7/4");
        const Q main_lower = (Q(1) - B / (Q(2) * Q(3, 2))) /
                             (Q(4) * Q(7, 4) * Q(5, 3));
        check(main_lower == Q(299, 3500), "[2.34]",
              "main-term endpoint coefficient = 299/3500");
        check(Q(1, 5) < power(Q(3, 2), 4) / factorial(4), "[2.34]",
              "(3/2)^4/4! > 1/5");
        polynomial_bound({{Q(10000), 2}}, B, Q(1), "[2.34]",
                         "10000*beta^2 <= 1 for beta<=1/100", true);
        check(Q(4) / Q(2000) == Q(1, 500), "[2.34]", "4/2000 = 1/500");
        check(Q(299, 3500) - Q(1, 500) == Q(73, 875), "[2.34]",
              "299/3500 - 1/500 = 73/875");
        check(Q(1, 12) < Q(73, 875), "[2.34]", "73/875 > 1/12");

        // Squaring removes all square roots in the Gaussian constants.
        check(Q(1) < Q(4) * power(Q(49, 300), 2) * Q(3) * pi_low, "[2.40]",
```

```
      "1/(2*sqrt(3*pi)) < 49/300");
check(Q(9) < Q(3) * pi_low, "[2.39]",
      "1/(3*sqrt(3*pi)) < 1/9");
check(Q(8) < power(pi_low, 2), "[2.39]", "2*sqrt(2) < pi");
check(B / Q(24) < Q(1), "[C error]", "beta/24 < 1");
check(Q(2) / (Q(1) - B / Q(24)) < pi_low, "[C error]",
      "exp(beta/24)/pi < 1/2 via the geometric-series bound");
polynomial_bound({{Q(5, 2), 3}}, B, Q(1, 400000), "[C error]",
                 "(5/2)*beta^3 <= 1/400000", true);
polynomial_bound({{Q(5, 6), 2}}, B, Q(1, 12000), "[C error]",
                 "(5/6)*beta^2 <= 1/12000", true);
check(Q(12) * (Q(49, 300) + Q(1, 400000)) < Q(2), "[2.35]",
      "12*(49/300 + 1/400000) < 2");
check(Q(12) * (Q(1, 9) + Q(1, 12000)) < Q(2), "[2.35]",
      "12*(1/9 + 1/12000) < 2");
check(Q(2) < Q(7), "[2.41]", "2*t <= 2 < 7 for 0<=t<=1");
check(Q(1) < Q(4), "[2.41]", "(1+t)/(4+t) < 1");
check(Q(3) / (Q(2) * alpha_low) < Q(1), "[2.41]",
      "3/(2*alpha) < 1");
check(Q(1) / (Q(3) * pi_low) < Q(1, 9), "[2.41]",
      "boundary coefficient 1/(3*pi) < 1/9");
check(Q(1, 4) < power(Q(3, 4), 2) / factorial(2), "[2.41]",
      "(3/4)^2/2! > 1/4");
polynomial_bound({{power(Q(4, 9), 2), 3}}, B, power(Q(1, 1000), 2),
                 "[2.41]", "(4/9)*beta^(3/2) < 1/1000");
polynomial_bound({{Q(1), 2}}, B, Q(1, 1000), "[2.41]",
                 "beta^2 < 1/1000");
check(Q(49, 300) + Q(2, 1000) < Q(1, 5), "[2.42]",
      "49/300 + 2/1000 < 1/5");
check(Q(12, 5) < Q(3), "[2.42]", "12/5 < 3");

check(Q(12) / (pi_low * (Q(1) - B / Q(24))) < Q(5), "[2.46]",
      "major-arc error coefficient < 5");
check(Q(24) / (Q(1) - B / Q(24)) < Q(25), "[2.47]",
      "minor-arc error coefficient < 25");
check(Q(5) + Q(25) == Q(30), "[2.47]", "combined mass coefficient = 30");
check(Q(4) / pi_low < Q(2), "[2.48]", "boundary coefficient 4/pi < 2");
check(Q(1) / ten21 < power(Q(3, 4), 20) / factorial(20), "[2.49]",
      "(3/4)^20/20! > 10^(-21)");
polynomial_bound({{Q(5) * power(ten21, 2), 29}}, b, Q(1), "[2.49a]",
                 "5*10^42*b_star^29 < 1");
polynomial_bound({{power(Q(30) * ten21, 2), 17}}, b, Q(1), "[2.49b]",
                 "(30*10^21)^2*b_star^17 < 1");
polynomial_bound({{power(Q(2) * ten21, 2), 21}}, b, Q(1), "[2.49c]",
                 "(2*10^21)^2*b_star^21 < 1");

// Values and derivative bounds for rho, tau, sigma and chi.
const Q xi0(0), z0 = Q(2) + xi0;
const Q rho0 = xi0 + Q(1) / z0;
const Q rho1 = Q(1) - Q(1) / power(z0, 2);
const Q tau0 = Q(3, 2) - Q(1) / (Q(2) * z0) -
               Q(1) / power(z0, 2);
const Q tau1 = Q(1, 2) / power(z0, 2) + Q(2) / power(z0, 3);
const Q sigma0 = rho0 + xi0 * tau0;
const Q chi0 = Q(2) * xi0 * rho0 + power(xi0, 2) * tau0;
check(tau0 == Q(1), "[derivatives]", "tau(0) = 1");
check(tau1 == Q(3, 8), "[derivatives]", "tau'(0) = 3/8");
check(sigma0 == Q(1, 2), "[derivatives]", "sigma(0) = 1/2");
check(rho1 + tau0 == Q(7, 4), "[derivatives]", "sigma'(0) = 7/4");
check(chi0 == Q(0), "[derivatives]", "chi(0) = 0");
check(Q(2) * rho0 == Q(1), "[derivatives]", "chi'(0) = 1");
const Q rho1_bound = Q(1) + Q(1, 4);
const Q rho2_bound = Q(2) / Q(8);
const Q tau_bound = Q(3, 2) + Q(1, 4) + Q(1, 4);
const Q tau1_bound = Q(1, 2) / Q(4) + Q(2) / Q(8);
const Q tau2_bound = Q(1, 8) + Q(6, 16);
check(rho1_bound == Q(5, 4), "[derivatives]", "sup|rho'| <= 5/4");
```

```
        check(rho2_bound == Q(1, 4), "[derivatives]", "sup|rho''| <= 1/4");
        check(tau_bound == Q(2), "[derivatives]", "sup|tau| <= 2");
        check(tau1_bound == Q(3, 8), "[derivatives]", "sup|tau'| <= 3/8");
        check(tau2_bound == Q(1, 2), "[derivatives]", "sup|tau''| <= 1/2");
        check(rho2_bound + Q(2) * tau1_bound + tau2_bound == Q(3, 2),
              "[derivatives]", "sup|sigma''| <= 3/2");
        check(Q(4) * rho1_bound + Q(2) * rho2_bound + Q(2) * tau_bound +
                      Q(4) * tau1_bound + tau2_bound == Q(23, 2),
              "[derivatives]", "sup|chi''| <= 23/2");

        polynomial_bound({{Q(2), 0}, {Q(1), 9}}, b, Q(3), "[nu1]",
                         "2 + b_star^9 < 3");
        const Q first_coefficient = Q(2, 3) * (Q(3, 8) * Q(3) + Q(1, 3));
        check(first_coefficient == Q(35, 36), "[2.43a]",
              "first signed error coefficient = 35/36");
        polynomial_bound({{Q(35, 36), 0}, {Q(2, 3), 9}, {Q(2), 8}}, b, Q(1),
                         "[2.43a]", "35/36 + (2/3)*b_star^9 + 2*b_star^8 < 1");
        polynomial_bound({{Q(2, 3), 0}, {Q(1), 1}}, b, Q(7, 10), "[2.52]",
                         "2/3 + b_star < 7/10");
        const Q second_coefficient = Q(2, 3) * (Q(7, 4) * Q(7, 10) + Q(1));
        check(second_coefficient == Q(89, 60), "[2.43b]",
              "second signed error coefficient = 89/60");
        polynomial_bound({{Q(89, 60), 0}, {Q(1, 3), 9}, {Q(2), 8}}, b, Q(2),
                         "[2.43b]", "89/60 + (1/3)*b_star^9 + 2*b_star^8 < 2");
        const Q third_coefficient = Q(2, 3) * (Q(7, 10) + Q(23, 4) * Q(4, 3));
        check(third_coefficient == Q(251, 45), "[2.43c]",
              "third signed moment coefficient = 251/45");
        polynomial_bound({{Q(251, 45), 0}, {Q(2), 8}}, b, Q(6), "[2.43c]",
                         "251/45 + 2*b_star^8 < 6");
        check(Q(12) * Q(2, 9) == Q(8, 3), "[2.44]",
              "fourth absolute-moment coefficient = 8/3");
        polynomial_bound({{Q(8, 3), 0}, {Q(1), 8}}, b, Q(3), "[2.44]",
                         "8/3 + b_star^8 < 3");

        const auto elapsed = std::chrono::duration_cast<std::chrono::microseconds>(
                                 std::chrono::steady_clock::now() - start).count();
        std::cout << (failures ? "COMPLETE_FAIL" : "COMPLETE_PASS")
                  << " checks=" << checks << " failed=" << failures
                  << " monotone_polynomial_checks=" << polynomial_checks
                  << " elapsed_us=" << elapsed << '\n';
        return failures ? 1 : 0;
    }
    catch (const std::exception &error)
    {
        std::cerr << "ERROR " << error.what() << '\n';
        return 2;
    }
}
```